Chapitre 2

Langages et Mots.

Sommaire.

1 Expressions régulières. **1**

2 Automates finis. **3**

3 Grammaires non contextuelles. **8**

3.1 Arbre d'analyse syntaxique. 9

Les propositions marquées de ★ sont au programme de colles.

Définition 1

On considère un alphabet Σ et on appelle mot une suite d'éléments de Σ.
Un élément de Σ est appelé une lettre.

Définition 2

On appelle langage sur Σ un ensemble de mots sur l'alphabet Σ.

1 Expressions régulières.

Définition 3

Les expressions régulières sur Σ sont définies inductivement.
— $\forall a \in \Sigma$, a est une expression régulière.
— ε et $\varnothing$ sont des expressions régulières.
— $e_1 \mid e_2$ est une expression régulière si e_1 et e_2 en sont.
— $e_1 e_2$ est une expression régulière si e_1 et e_2 en sont.
— e_1^* aussi.

Définition 4

À chaque expression régulière e, on souhaite associer un langage $L(e)$.
Il est défini inductivement :
— $L(a) = \{a\}$.
— $L(\varnothing) = \varnothing$.
— $L(\varepsilon) = \{\varepsilon\}$ le mot vide.
— $L(e_1 e_2) = L(e_1)L(e_2)$.
— $L(e_1 \mid e_2) = L(e_1) \cup L(e_2)$.
— $L(e_1^*) = \bigcup_{i=0}^{+\infty} L(e_1)^i$, appelé étoile de Kleene.

Définition 5

Deux expressions régulières e_1 et e_2 sont équivalentes si $L(e_1) = L(e_2)$.

Exemple 6

Si e est une expression régulière, alors :
— $e \mid \varnothing \sim e$.
— $e\varnothing \sim \varnothing$.
— $\varnothing^* = \varepsilon$.

Proposition 7

Si e est une expression régulière, il existe e' qui n'utilise pas $\varnothing$ telle que $e \sim e'$ ou $e \sim \varnothing$.

Preuve :

– Si $e = a$, avec $a \in \Sigma$, alors $e' = a$ convient.
– Si $e = \varepsilon$, alors $e' = \varepsilon$ convient.
– Si $e = \varnothing$, alors n'importe quel e' convient.
– Si $e = e_1 \mid e_2$, par induction, $e_1 \sim e_1'$ ou $e_1 \sim \varnothing$, de même pour e_2.
— Si $e_1 \sim e_1'$ et $e_2 \sim e_2'$, alors $e \sim e_1' \mid e_2'$ n'utilise pas $\varnothing$.
— Si $e_1 \sim e_1'$ et $e_2 \sim \varnothing$, alors $e \sim e_1'$.
— Si $e_1 \sim \varnothing$ et $e_2 \sim \varnothing$, alors $e \sim \varnothing$.
– Si $e = e_1 \cdot e_2$, alors $e \sim \varnothing$ ssi $e_1 \sim \varnothing$ ou $e_2 \sim \varnothing$.
– Si $e = e_1^*$, alors $e = \varepsilon$ si $e_1 \sim \varnothing$, sinon $e = e_1'^*$ si $e_1 \sim e_1'$.

Corrolaire 8

À part pour décrire le langage vide, le symbole $\varnothing$ est inutile dans la définition des expression régulières.

Proposition 9

Si e est une expression régulière sans $\varnothing$, alors il existe e' sans ε tel que :
— $e' \sim e$ ou $e \sim e' \mid \varepsilon$ ou $e \sim \varepsilon$.

Exemple 10

Trouver des expressions régulières pour :
— L'ensemble des mots de $\{a, b\}$ qui contiennent un nombre pair de a.
— L'ensemble des mots de $\{a, b\}$ qui ne contient pas deux a d'affilée.
— L'ensemble des mots de taille 3.

Solution :

1. $(ab^*a \mid b^*)^*$
2. $a \mid (b \mid ab)^*(a \mid \varepsilon)$.
3. $(a \mid b)(a \mid b)(a \mid b)$.

Proposition 11

On a $|\Sigma|^k$ mots de taille k.

Proposition 12

Si L est un langage fini de taille finie, il existe une expression régulière e telle que $L(e) = L$.

Définition 13

On dit d'un langage qu'il est régulier s'il existe une expression qui le caractérise.

Proposition 14

Il existe des langages non réguliers.

Preuve :

Un langage peut être vu comme un élément de $\{0, 1\}^{\mathbb{N}}$. La valeur i indique si le $i$ème mot appartient au langage.

Définition 15

On définit $\mathrm{Reg}(\Sigma)$ comme l'ensemble des langages réguliers sur Σ.

Proposition 16

On a $\mathrm{Reg}(\Sigma)$ le plus petit ensemble de langages contenant $\{\varepsilon\}, \varnothing, \{a\}$ qui soit stable par union, concaténation et étoile de Kleene.

Preuve :

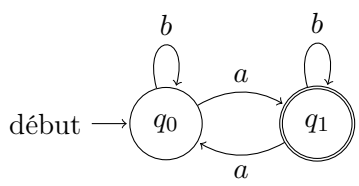
Montrons que $\mathrm{Reg}(\Sigma)$ vérifie ces propriétés.
— $\{\varepsilon\} \in \mathrm{Reg}(\Sigma)$ car caractérisé par ε, de même pour $\{a\}$ et $\varnothing$.
— Soient $L_1, L_2 \in \mathrm{Reg}(\Sigma)$. Il existe e_1, e_2 deux expressions régulières telles que $L(e_1) = L_1$ et $L(e_2) = e_2$.
Alors $L(e_1 \mid e_2) = L(e_1) \cup L(e_2) = L_1 \cup L_2$; $L(e_1 e_2) = L_1 L_2$ et $L(e_1^*) = L_1^*$.
— Soit X un ensemble de langages vérifiant ces propriétés. On va réaliser une induction sur les langages réguliers.
Soit $L \in \mathrm{Reg}(\Sigma)$ et e une expression régulière qui le caractérise.
— Si $e \in \{\varnothing, \varepsilon, a\}$, alors $L \in X$.
— Si $e = e_1 \mid e_2$, alors $L_1, L_2 \in X$, puis $L(e) = L(e_1) \cup L(e_2) \in X$.
— Si $e = e_1 e_2$, alors $L_1, L_2 \in X$, puis $L(e) = L(e_1)L(e_2) \in X$.
— Si $e = e_1^*$, alors $L_1 \in X$, puis $L(e) = \bigcup_k L(e_i)^k$.
Donc $\mathrm{Reg}(\Sigma) \in X$.

2 Automates finis.

Définition 17

Un automate est un quintuplet $(\Sigma, Q, q_0, F, \delta)$ où :
— Σ est l'alphabet.
— Q est l'ensemble des états.
— $q_0 \in Q$ est l'état initial.
— $F \subset Q$ est l'ensemble des états finaux.
— $\delta : Q \times \Sigma \to Q$ est la fonction de transition.

Exemple 18



Si un mot est accepté, il contenait un nombre impair de a, sinon il en contenait un nombre pair.
Alors : $\Sigma = \{a, b\}$, $Q = \{q_0, q_1\}$, $F = \{q_1\}$ et :

$Q \setminus \Sigma$	a	b
q_0	q_1	q_0
q_1	q_0	q_1

Définition 19

On étend la défintiion de δ de la façon suivante :
1. $\forall q \in Q,\ \delta^*(q, \varepsilon) = q$.
2. $\forall q \in Q,\ \forall a \in \Sigma,\ \forall m \in \Sigma^*,\ \delta^*(q, am) = \delta^*(\delta(q, a), m)$.

Un mot m est dit accepté par l'automate $A = (\Sigma, Q, q_0, F, \delta)$ si $\delta^*(q_0, m) \in F$.

Définition 20

Si A est un automate, on note $L(A)$ l'ensemble des mots acceptés par A, le langage reconnu par A.

Exemple 21

Montrons que $L_A = \{m \in \Sigma^*,\ |m|_a \equiv 1[2]\}$, où $|m|_a$ est le nombre de a dans m.

Solution :

Par récurrence, $\mathcal{P}_n$: « $\forall q_i \in Q,\ \forall m \in \Sigma^*,\ \delta^*(q_i, m) = q_j$ où $j \equiv i + |m|_a[2]$ ».
Initialisation. On a $\delta^*(q_i, \varepsilon) = q_i$ et $i \equiv i + 0[2]$.
Hérédité. Soit $n \in \mathbb{N}$ tel que $\mathcal{P}_n$. Soit $m = cm'$ de taille $n+1$, alors $\delta^*(q_i, m) = \delta^*(\delta(q_i, c), m')$.
– Si $c = a$, alors $\delta(q_i, c) = q_j$ avec $j \equiv i + 1[2]$, donc $\delta^*(q_j, m') = q_{j'}$ où $j' \equiv j + |m'|_a[2] = i + |m|_a[2]$.
– Si $c = b$, alors $\delta(q_i, c) = q_i$ donc $\delta^*(q_i, m) = \delta^*(q_i, m') = q_j$ avec $j \equiv i + |m'|_a[2] = i + |m|_a[2]$.

On traite le cas $q_i = q_0$: $\forall m \in \Sigma^*,\ \delta^*(q_0, n) = q_j$ avec $j = |m|_a[2]$.
Le mot est accepté ssi $j = 1$ ssi $|m|_a \equiv 1[2]$ donc $L_A = \{m \in \Sigma^*,\ |m|_a \equiv 1[2]\}$.

On pourrait définir $\widetilde{\delta^*}(q, \varepsilon) = q$ et $\widetilde{\delta^*}(q, ma) = \delta(\widetilde{\delta^*}(q, m), a)$.

Définition 22

On dit d'un état q qu'il est accessible s'il existe un mot m tel que $\delta^*(q_0, m) = q$.

Définition 23

Un état est dit co-accessible s'il existe un mot m tel que $\delta^*(q, m) \in F$.

Définition 24

Un état est utile s'il est accessible et co-accessible.
Un automate est dit émondé si tous ses états sont utiles.
La fonction de transition n'est définie que partiellement.
Le langage des mots acceptés est celui des mots m tels que $\delta^*(q_0, m)$ est défini et dans F.
Si la fonction de transition est définie sur tout $Q \times \Sigma$, on dit que l'automate est complet.

Définition 25

Pour compléter un automate, on peut ajouter un état q_p et on complète la fonction de transition :

$$\forall q \in Q,\ \forall a \in \Sigma, \text{si } \delta(q, a) \text{ n'est pas défini, } \delta(q, a) := q_p.$$

$$\forall a \in \Sigma,\ \delta(q_p, a) := q_p.$$

Définition 26: Automate non déterministe.

Un automate non déterministe est un quintuplet : $(\Sigma, Q, I, F, \delta)$, et $\delta : Q \times \Sigma \to \mathcal{P}(Q)$.
Un mot est accepté par l'automate s'il existe un chemin d'un état initial à un état final qui soit étiqueté par m.
On cherche à étendre δ aux mots de Σ^* :
— $\forall q \in Q,\ \delta^*(q, \varepsilon) = \{q\}$.
— $\forall q \in Q,\ \forall a \in \Sigma, \forall m \in \Sigma^*, \delta^*(q, am) = \bigcup_{q_i \in \delta(q,a)} \delta^*(q_i, m)$.

Proposition 27

On peut transformer un automate déterministe en non-déterministe.
Il suffit de prendre $I = \{q_0\}$ et $\delta'(q, a) = \{\delta(q, a)\}$.

L'opération inverse est possible.

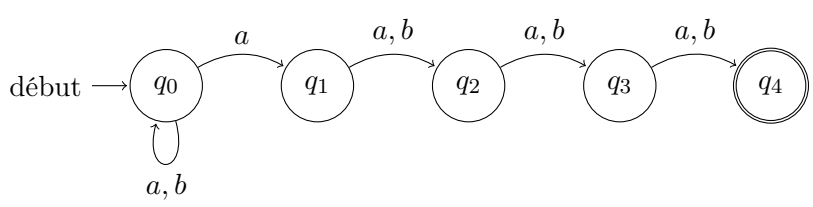
Définition 28

Soit $A = (\Sigma, Q, I, F, \delta)$ non déterministe, on définit A' de la façon suivante :
— Σ est le même.
— $Q' = \mathcal{P}(Q)$.
— $q_0 = I$.
— $F' = \{E \in \mathcal{P}(Q),\ E \cap F \neq \varnothing\}$.
— $\delta'(E, a) = \bigcup_{q \in E} \delta(q, a)$. (l'union des états atteignables par une lettre a depuis un état de E).
On appelle ce procédé la déterminisation d'un automate.
En pratique, on ne fait apparaître que les éléments de $\mathcal{P}(Q)$ par lesquels on passe.
L'automate produit par ce procédé a un nombre d'états exponentiel en celui de l'automate initial.

Exemple 29

On considère L_n le langage des mots de $\{a, b\}^*$ qui contiennent un a n lettres avant la fin.
Construire un automate non-déterministe, puis déterministe qui reconnaît les mots de L_3.

Solution :



Proposition 30

Tout automate déterministe reconnaissant L_n a au moins 2^{n+1} états.

Preuve :

On considère u et v deux mots de Σ^{n+1}. Montrons que $u \neq v \Rightarrow \delta^*(q_0, u) \neq d^*(q_0, v)$.
Supposons $u \neq v$, alors $\exists (u_1, u_2), (v_1, v_2),\ u = u_1 a u_2$ et $v = v_1 b v_2$, avec $|u_1| = |v_1|$ et $|u_2| = |v_2|$.
Soit w le mot $a^{n-|u_2|}$, alors $uw = u_1 a u_2 w$ et $|u_2| + |w| = n$ donc uw est accepté, et vw ne l'est pas.
Alors $\delta^*(q_0, uw) \in F$ et $\delta^*(q_0, vw) \notin F$, or $\delta^*(q_0, uw) = \delta^*(\delta^*(q_0, u), w)$ et $\delta^*(q_0, vw) = \delta^*(\delta^*(q_0, v), w)$.
On a donc $\delta^*(q_0, u) \neq \delta^*(q_0, v)$, donc δ^* est injective de Σ^{n+1} vers Q donc $|Q| \geq |\Sigma^{n+1}| = 2^{n+1}$.

Définition 31

Le passage au miroir de L, noté $\overline{L}$ est l'ensemble $\{\overline{w},\ w \in L\}$, où $\overline{w}$ est w lu à l'envers.

Proposition 32

L'ensemble des langages reconnus par des automates est stable par intersection.

Preuve :

Soient $A_1 = (\Sigma_1, Q_1, q_0^1, F_1, \delta_1)$, $A_2 = (\Sigma_2, Q_2, q_0^2, F_2, \delta_2)$ deux automates déterministes sans perte de généralité.
Soient L_1 et L_2 les langages reconnus par A_1 et A_2. On cherche un automate reconnaissant $L_1 \cap L_2$.
Soit $w \in L_1 \cap L_2$: $\delta^*(q_0^1, w) \in F_1$ et $\delta^*(q_0^2, w) \in F_2$. On pose :

$$\Sigma = \Sigma_1 \cup \Sigma_2,\ Q = Q_1 \times Q_2,\ q_0 = (q_0^1, q_0^2),\ F = F_1 \times F_2, \quad \text{et} \quad \delta : \begin{cases} Q_1 \times Q_2 \times \Sigma & \to & Q_1 \times Q_2 \\ (q_1, q_2, a) & \mapsto & (\delta_1(q_1, a), \delta_2(q_2, a)) \end{cases}$$

Par récurrence sur la taille de w, $\delta^*((q_0^1, q_0^2), w) = (\delta_1^*(q_0^1, w), \delta_2^*(q_0^2, w))$.

On définit $A = (\Sigma, Q, q_0, F, \delta)$. On a alors :

$$\begin{aligned} w \in L(A) &\iff \delta^*((q_0^1, q_0^2), w) \in F_1 \times F_2 \\ &\iff (\delta_1^*(q_0^1, w), \delta_2^*(q_0^2, w)) \in F_1 \times F_2 \\ &\iff \delta_1^*(q_0^1, w) \in F_1 \ \text{ et } \ \delta_2^*(q_0^2, w) \in F_2 \\ &\iff w \in L_1 \cap L_2. \end{aligned}$$

Donc $L(A) = L_1 \cap L_2$.

Proposition 33

L'ensemble des langages reconnus par des automates est stable par passage au miroir.

Preuve :

Soit $A = (\Sigma, Q, q_0, F, \delta)$ un automate déterministe et complet qui reconnaît L.
On définit $A' = (\Sigma, Q, F, q_0, \delta')$, avec $\forall (q, a) \in Q \times \Sigma,\ \delta(q, a) = q' \Rightarrow q \in \delta'(q', a)$ (on retourne les flèches).
Par récurrence sur la taille de w, $\forall q \in Q,\ \forall w \in \Sigma^*,\ \delta^*(q, w) = q' \iff q \in \delta'^*(q', \overline{w})$.

$$\begin{aligned} w \in L &\iff \exists q_F \in F,\ \delta^*(q_0, w) = q_F \iff q_0 \in \delta'^*(q_F, \overline{w}) \\ &\iff q_0 \in \bigcup_{q_F \in F} d^*(q_F, \overline{w}) \iff \overline{w} \in L(A'). \end{aligned}$$

Donc $L(A') = \overline{L}$.

Définition 34: Algorithme d'élimination des états.

$\boxed{1.}$ Ajouter α et ω deux nouveaux états, α nouvel initial, ω nouvau final.
L'automate ainsi produit a des transitions étiquetées avec des expressions régulières.
$\boxed{2.}$ Éliminer un état, différent de α et ω, et reporter ses transitions.
$\boxed{3.}$ Répéter jusqu'à ce qu'il ne reste que α et ω.
L'expression régulière finale, reliant α et ω caractérise le langage reconnu par l'automate.

Justification :

L'algorithme termine car $|Q|$ est un variant. Il est correct car les langages reconnus par les automates intermédiaires sont les mêmes.

Définition 35

Un automate est dit généralisé si ses transitions sont étiquetées par des expressions régulières.

Définition 36

Un mot m est accepté par un automate généralisé s'il existe un chemin d'un état initial vers un état final étiqueté par une expression régulière e telle que $m \in L(e)$.

Définition 37

Soit L un langage, on définit :
— $P(L) = \{e \in \Sigma,\ \exists n \in \Sigma^*,\ en \in L\}$, l'ensemble des préfixes de L.
— $D(L) = \{e \in \Sigma,\ \exists n \in \Sigma^*,\ ne \in L\}$, l'ensemble des suffixes de L.
— $F(L) = \{u \in \Sigma^2,\ \exists n_1, n_2 \in \Sigma^*,\ n_1 u n_2 \in L\}$, l'ensemble des facteurs de L de taille 2.

Définition 38

Un langage est dit local si $L \setminus \{\varepsilon\} = (P(L)\Sigma^* \cap \Sigma^* D(L)) \setminus (\Sigma^*(\Sigma^2 \setminus F(L))\Sigma^*)$.
Ce sont les mots qui commencent et finissent comme un mot de L, et qui ne possèdent pas de facteur de taille 2 qui n'est pas un facteur de L.

Proposition 39

Si un langage est local, il est reconnaissable par un automate.

Preuve :

Soit $Q = \Sigma \cup \{\varepsilon\}$, $q_0 = \varepsilon$ et $F = D(L)$, $\forall e \in P(L),\ \delta(q_0, e) = e$, $\forall e_1 e_2 \in F(L),\ \delta(e_1, e_2) = e_2$.
q représente la dernière lettre lue. Si on a aucune lettre, on est en ε, et pour accepter un mot, la prochaine lettre lue doit appartenir à $D(L)$.
Si on est en e_1, la prochaine lettre lue doit former un élément de $F(L)$ lorsqu'elle est concaténée à e_1.
Cet automate reconnaît le langage $(P(L)\Sigma^* \cap \Sigma^* D(L)) \setminus (\Sigma^*(\Sigma^2 \setminus F(L))\Sigma^*)$.
Si L est local, on peut construire cet automate pour reconnaître $L \setminus \{e\}$.
Si $\varepsilon \in L$, on rajoute simplement ε aux états finaux.

Définition 40

Une expression régulière est dite linéaire si chaque lettre de Σ apparaît au plus une fois.

Proposition 41

Si e est une expression régulière linéaire, alors $L(e)$ est un langage local.

Preuve :

Par induction :
— $\varnothing$ ou ε : $P(L) = F(L) = D(L) = \varnothing$.
— a : $P(L) = D(L) = \{a\}$ et $F(L) = \varnothing$.
— e_1e_2 : $P(L) = P(L_1) \cup P(L_2)$ si $\varepsilon \in L_1$, sinon $P(L_1)$, pareil pour $D(L)$.
Traitons le cas où $\varepsilon \notin P(L_1)$, et $\varepsilon \notin D(L_2)$. On a m commence par une lettre de $P(L_1)$ et termine par $D(L_2)$.
Alors $m = m_1...m_n$, et $\exists i$ tel que m_i est une lettre utilisée pour e_1, et m_{i+1} est utilisée pour e_2.
Alors $n_i \in D(L_i)$ et $n_{i+1} \in P(L_2)$, et $\forall k < i,\ m_km_{k+1} \in F(L_1)$, et $\forall k \in [\![i+1, n-1]\!]$, $m_km_{k+1} \in F(L_2)$.
On a $m_im_{i+1} \in D(L_1)P(L_2)$ puis $m_1...m_i \in L_1 \setminus \varepsilon$ par induction, L_1 étant local.
Alors $m_{i+1}...m_n \in L_2 \setminus \{\varepsilon\}$ par induction et $m \in L_1L_2 = L(e)$.

Définition 42: Algorithme de Berry-Sethi.

Algorithme 1 : Algorithme de Berry-Sethi.

Entrées : Une expression régulière e.
1 - Linéariser e en e'.
2 - Calculer $D(L)$, $P(L)$ et $F(L)$.
3 - Construire l'automate (39).
4 - Délinéariser.
5 - Déterminiser.

Corrolaire 43

Tout langage caractérisé par une expression régulière est reconnaissable par un automate.

Corrolaire 44

Les langages réguliers sont stables par intersection et passage au miroir.

Exercice 1

Les langages réguliers sont stables par complémentaire.

Théorème 45: de Kleene.

Les langages reconnus par des automates sont exactement les langages réguliers.

Preuve :

$\boxed{\Rightarrow}$ Prouvé par l'algorithme d'élimination des états.
$\boxed{\Leftarrow}$ Prouvé par l'algorithme de Berry-Sethi.

Proposition 46

Un autre preuve aurait été de montrer que les langages reconnus par des automates sont stables par union, concaténation, et étoile de Kleene, et qu'ils contiennent $\varepsilon, \varnothing$ et a.

Définition 47: Éliminations d'ε-transitions.

Pour la cloture arrière, si q_2 était un état final, alors q_1 en devient un aussi.
Pour la cloture avant, si q_2 était un état initial, alors q_3 en devient un aussi.
Cloture avant : $q_1 \to_a q_2 \to_\varepsilon q_3$ devient $q_1 \to_a q_2$ et $q_1 \to_a q_3$.
Cloture arrière : $q_1 \to_\varepsilon q_2 \to_a q_3$ devient $q_1 \to_a q_3$ et $q_2 \to_a q_3$.

Théorème 48: Lemme de l'étoile.

Soit L un langage régulier.

$$\forall m \in L,\ |m| > |Q| \Rightarrow \exists x, y, z \in L,\ m = xyz,\ |xy| < |Q|,\ |y| > 0,\ \forall k \in \mathbb{N},\ xy^kz \in L$$

Preuve :

Il existe un automate $(\Sigma, Q, q_0, F, \delta)$ déterministe et complet qui reconnaît L.
Soit $m \in L$ tel que $|m| > |Q|$, on écrit $m = m_1...m_n$ les différentes lettres de m.
Soient $q_1, ..., q_n$ les états par lesquels on passe. Par principe des tiroirs, $\exists i, j \in [\![1, |Q|]\!],\ i \neq j$ et $q_i = q_j$.
On pose $x = m_1...m_i$, $y = m_{i+1}...m_j$ et $z = m_{j+1}...m_n$, alors $m = xyz$, $xy^kz \in L$, $|y| > 0$ et $|xy| \leq |Q|$.

Corrolaire 49

Par contraposée, si un langage ne vérifie pas le lemme de l'étoile, il n'est pas régulier.

Exemple 50

Le langage $L = \{a^n b^n,\ n \in \mathbb{N}\}$ n'est pas régulier.

Solution :

Par l'absurde, supposons qu'il vérifie le lemme de l'étoile : il existe N...
On pose $m = a^N b^N$, $\exists x, y, z \in L,\ m = xyz,\ |xy| < N,\ |y| > 0$ et $\forall k \in \mathbb{N},\ xy^k z \in L$.
Alors $\exists i \in \mathbb{N},\ xy = a^i$ puis $\exists j \in \mathbb{N},\ y = a^j$ et $\forall k \in \mathbb{N},\ xy^k z \in L$.
Alors $a^{i-j}(a^j)^k a^{N-i} b^N \in L$, c'est vrai pour $k = 0$, donc $a^{i-j} a^{N-i} b^N = a^{N-j} b^N \in L$, contradiction car $j > 0$.

Méthode

1. Supposer le langage régulier.
2. Considérer un mot qui convient.
3. Déduire de la décomposition la forme des x, y, z.
4. Aboutir à une contradiction.

Proposition 51

Il existe des langages non-réguliers qui vérifient le lemme de l'étoile.

Exercice 2

Montrer que $L = \{a^{(2^n)},\ n \in \mathbb{N}\}$ n'est pas régulier.

Exemple 52

Montrer que $L = \{a^p,\ p \text{ premier}\}$ n'est pas régulier.

Solution :

Par l'absurde, supposons qu'il vérifie le lemme de l'étoile : il existe N...
Soit p plus petit premier plus grand que N et $m = a^p$. Alors $\exists x, y, z \in L,\ xyz = a^p,\ |xy| < N,\ |y| > 0,\ xy^k z \in L$.
On a $\exists j > 0,\ y = a^j$. Alors $\forall k \in \mathbb{N},\ a^{kj} a^{p-j} \in L$. Alors pour $k = p + 1$, $a^{(j+1)p} \in L$, absurde.

3 Grammaires non contextuelles.

Définition 53

Une **grammaire** est un quadruplet $(\mathcal{V}, S, \Sigma, R)$ où :
— $\mathcal{V}$ est l'ensemble des symboles **non-terminaux**. (variables).
— S est le **symbole initial**, avec $S \in \mathcal{V}$.
— Σ est l'ensemble des symboles terminaux.
— R est l'ensemble des **règles de production**, composé d'éléments de $\mathcal{V} \times (\Sigma \cup \mathcal{V})^*$.
Notation. Une règle est notée $(X, m) : X \to M$.

Définition 54: Dérivation immédiate.

On parle de **dérivation** selon la grammaire G :

$$m_1 \Rightarrow m_2 \iff \exists \alpha, \beta, \gamma, X,\ m_1 = \alpha X \beta \text{ et } m_2 = \alpha\gamma\beta, \text{ avec } (X, \gamma) \in \mathcal{R}.$$

Dans ce cas, on parle de dérivation immédiate.
S'il n'y a pas de non-terminaux dans α (resp. β), alors la dérivation est dite **gauche** (resp. **droite**).
Remarque. On parle de grammaire non-contextuelle car la dérivation est indépendante du contexte.

Définition 55: Dérivation.

On note $\Rightarrow^*$ la dérivation entre deux mots définie ainsi :
1. $\forall m,\ m \Rightarrow^* m$.
2. $\forall m_1, m_2, m_3,\ (m_1 \Rightarrow^* m_2 \text{ et } m_2 \Rightarrow m_3)$ implique $m_1 \Rightarrow^* m_3$.
3. $\forall m_1, m_2,\ m_1 \Rightarrow m_2$ implique $m_1 \Rightarrow^* m_2$.

On a donc que $\Rightarrow^*$ est la fermeture réflexive et transitive de $\Rightarrow$.

Définition 56: Langage engendré.

Si G est une grammaire $(\mathcal{V}, S, \Sigma, \mathcal{R})$, on note $\mathscr{L}(G) = \{m \in \Sigma^*,\ S \Rightarrow^* m\}$ le langage engendré par la grammaire.

Exemple 57

Montrer que $\mathscr{L}(G) = \{a^n b^n,\ n \in \mathbb{N}\}$.

Solution :

$\boxed{\supset}$ Par récurrence sur $n \in \mathbb{N}$.
Initialisation. On a bien $S \Rightarrow^* \varepsilon$ car G contient la règle (S, ε).
Hérédité. Soit $n \in \mathbb{N}$ tel que la propriété soit vraie au rang n.
On a $S \Rightarrow^* a^n b^n$ par hypothèse de récurrence, et $S \Rightarrow aSb \Rightarrow^* aa^n b^n b = a^{n+1} b^{n+1}$.
Par récurrence, la propriété est vraie pour tout $n \in \mathbb{N}$.
$\boxed{\subset}$ Montrons que pour k dérivations, $S \Rightarrow^k n$ implique $n = a^{k-1} b^{k-1}$.
Initialisation. C'est vrai pour $k = 1$.
Hérédité. Soit $k \in \mathbb{N}^*$ tel que la propriété soit vraie au rang k.
La première dérivation immédiate est $S \Rightarrow aSb$.
Il reste k dérivations immédiates : $S \Rightarrow^{k+1} m$ donc $aSb \Rightarrow^k m$ donc $\exists m_1,\ m = am_1b$ et $S \Rightarrow^k m_1$.
Par hypothèse de récurrence, $m_1 = a^{k-1} b^{k-1}$ donc $m = a^k b^k$.

Théorème 58

Les langages réguliers sont inclus dans les langages engendrés par des grammaires non contextuelles.

Preuve :

Soit $\mathscr{L}$ régulier, il est reconnu par un automate $A = (\Sigma, Q, q_0, F, \delta)$.
On considère la grammaire $G = (\mathcal{V}, S, \Sigma, R)$ définie ainsi :
$\hookrightarrow$ $\mathcal{V} = Q$, $S = q_0$, $\Sigma = \Sigma$ et $\mathcal{R}$ défini tel que :

$$\forall q \in Q,\ \forall a \in \Sigma,\ q \to a\delta(q, a) \quad \text{et} \quad \forall q \in F,\ q \to \varepsilon.$$

(La preuve n'a pas été terminée, on l'a fait sur un exemple.)

3.1 Arbre d'analyse syntaxique.

Exemple 59

Soit G la grammaire définie par :
— $\mathcal{V} = \{F\}$, $S = F$ et $\Sigma = \{\top, \bot, \wedge, \vee, \neg, (,)\}$.
— $R : F \to \bot \mid \top, \quad F \to F \wedge F \mid F \vee F \mid \neg F \mid (F)$.
Alors G engendre les formules propositionnelles n'utilisant pas de littéraux.

Définition 60

Si un mot possède deux arbres d'analyse syntaxique différents, on dit de la grammaire qu'elle est ambigüe.
Pour éviter qu'un même mot ne puisse être compris de plusieurs manières différentes, on souhaite ne pas avoir d'ambigüité.

Exemple 61

On peut donc modifier $\mathcal{R}$:
— $F \to F_1 \vee F \mid F_1$.
— $F_1 \to F_2 \wedge F_1 \mid F_2$.
— $F_2 \to \neg F_2 \mid \bot \mid \top \mid (F)$.
On a fait le choix de prioriser les $\wedge$ sur les $\vee$.

Proposition 62: Admis.

Il existe des langages qui ne sont pas engendrés par des grammaires non contextuelles.

Proposition 63: Admis.

Il existe des langages engendrés par des grammaires non contextuelles tels que la grammaire est forcément ambigüe.

Exemple 64

Le langages des palindromes sur $\{a, b\}$ et le langage de Dyck sur $\{(,)\}$.

Solution :

Dyck : $S \to \varepsilon$, $S \to S(S)$; et $S \to \varepsilon$, $S \to (S)$, $S \to SS$.

Palindromes : $S \to \varepsilon \mid a \mid b$;

Définition 65: Forme normale de Chomsky

Une grammaire G est sous forme normale de Chomsky si toutes ses règles sont d'une des formes suivantes :
— $X \to YZ$.
— $X \to x$.
— $S \to \varepsilon$.
avec S le symbole initial, X, Y, Z des non-terminaux, Y, Z différents de S et x un terminal.
Remarque. La première règle augmente la taille du mot, la deuxième augmente le nombre de terminaux.

Proposition 66

Si m est de taille n et engendré par G, alors sa dérivation est composée de n règles du second type et $n-1$ règles du premier type. Il y a exactement $2n-1$ dérivations.

Les grammaires des langages vérifient certaines propriétés qui permettent généralement une analyse en $O(n)$.

Proposition 67

Si G est une grammaire non contextuelle, il existe G' sous forme normale de Chomsky telle que $L(G) = L(G')$.

Preuve :

Soit G une grammaire.
Première étape : éviter que le symbole initial apparaisse à droite.
On crée un nouveau symbole pour le remplacer à droite.
Deuxième étape : limiter la taille des règles de droite.
On décompose les règles de trop grande taille, par exemple: $X \to AXB$ devient $X \to AC$ et $C \to XB$.
Troisième étape : remplacer les règles de taille 2 avec des symboles terminaux.
On remplace le symbole terminal par une règle qui renvoie sur lui.
Quatrième étape : éliminer les règles de la forme $X \to \varepsilon$ avec X non initial.
Si $X \to \varepsilon$ et $A \to X$, alors $A \to \varepsilon$, si $A \to XY$, alors $A \to Y$.
Cinquième étape : remplacer les non-terminaux isolés.
Si $X \to Y$, et $Y \to a$, alors $X \to a$. Si $Y \to AB$, alors $X \to AB$.